Graph Analytics and Network Science

Sujwal Khadka

Westcliff University

DCS 401: Query Design and Analysis

Professor Vaidhya

October 3, 2026

Graph Analytics and Network Science

Graph analytics and network science are crucial tools used to analyze complex datasets and understand their relationships and connections. A graph consists of nodes and edges: Nodes can be people, computers, products, organizations, etc., and edges are the relationships between nodes. Graph Analytics differs from traditional data analysis by analyzing the connections between data records as well as the data itself. Graph analytics can be applied to various fields like social networks, cybersecurity, recommendation systems, transportation, fraud detection, and business intelligence. This assignment will concentrate on two important graph analytics techniques, centrality measures and community detection. We will use Python and the NetworkX library in Google Colab to implement these concepts in the practical section.

Centrality Measures

Centrality measures help determine the most important or influential nodes in a graph. There are different kinds of centrality and they measure the importance in different ways. Degree centrality is defined as the number of direct connections a node has. A hub (node with multiple direct connections) can be viewed as very influential or active. Betweenness centrality is the number of shortest paths that pass through a node between other nodes. High betweenness nodes tend to be "bridges" between regions of a network. Closeness centrality is defined as how easily a node can reach all other nodes in the network.

An advantage of centrality analysis is its ability to make it easier to find important entities in large networks. For instance, it is capable of identifying influential social media users, key employees within an organization, key servers within a computer network, or key transportation nodes, etc. But there are also limitations of centrality measures. Various centrality measures may identify different nodes as being the most central. A node can, for example,

connect many nodes and may not necessarily link nodes from different groups. Certain centrality methods are also susceptible to having large computation time when applied to very large networks.

In the real world, centrality is exemplified by social media. A company could build a network where the nodes are the users and friendships/interactions are the edges. The degree centrality could identify users who have many immediate connections, while betweenness centrality could identify users who are the bridges between groups. These key users might be helpful for marketing efforts, information sharing, or product promotion.

Community Detection

Community detection is a graph analytics method that aims to find communities of more densely interconnected nodes as defined by a threshold, which is typically chosen in a subjective way (Cheramangalath et al., 2020). The groups are typically referred to as communities or clusters. For instance, a social media network might have groups of users who share common interests, friendships, jobs or geographical areas. These are recognized by community detection algorithms, which are based on the relationships between nodes.

There are a number of benefits to community detection. It can uncover underlying structures in a large and complex graph and can simplify a graph. Community detection can be applied in businesses to segment customers, provide recommendations, analyze fraud, and build targeted marketing campaigns. But there are drawbacks, too. In some real-world networks, there can be multiple communities, and different algorithms can yield different communities. Furthermore, very large graphs can consume a lot of computational resources.

One can think of the case of e-commerce. Customers may be represented as nodes and relationships established when customers buy similar products (Hagberg, 2024). Based on that, a community detection could detect clusters of customers that love games, sports, technology, or fashion. These groups can then be used by a company to make more customized product suggestions.

Implementing with Python and NetworkX.

To illustrate the theoretical concepts, a theoretical social network dataset was analysed in Python in Google Colab. The data includes nodes A, B, ..., X (not all of them are included), labeled nodes, and relationships between them. The Pandas library was used to load the dataset, NetworkX was used to create and analyze the graph, and Matplotlib was used to visualize the network.

Loading the Dataset

There are 3 columns: Source, Target, and Weight. Source and Target are two nodes connected together, with the Weight indicating how well they are related.

```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("week6_graph_edges.csv")

df.head()
```

	Source	Target	Weight
0	A	B	5
1	A	C	4
2	A	D	3
3	B	C	5
4	B	D	4

Fig 1: Loading the dataset

Creating and Visualizing the Graph

The dataset was converted into graph using Network X

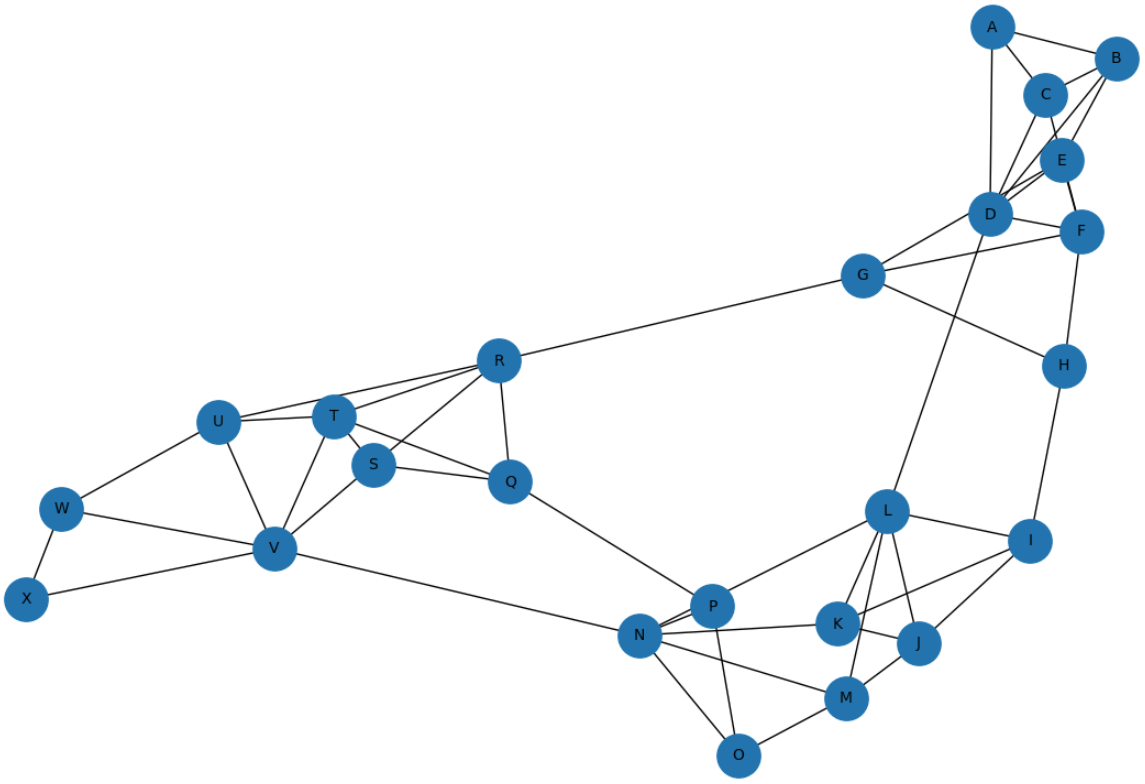


Fig 2: Visualization of the hypothetical social network using NetworkX

The visualization represents users as nodes and their relationships as edges. Some areas of the network appear more connected than others, suggesting that different groups may exist within the overall graph.

Graph Transversal Using BFS and DFS

Breadth First Search and Depth First Search were also applied to demonstrate graph transversal BFS traverses the network level-wise starting with nodes which have direct links to the starting node (Jiang, 1991). DFS backtracks to another path after traversing as far as it can along one path. These methods can be helpful in understanding relationships, and how to traverse a graph.

Centrality Analysis

Three indices of centrality were computed: closeness centrality, betweenness centrality and degree centrality.

```
degree_centrality = nx.degree_centrality(G)

degree_df = pd.DataFrame(
    degree_centrality.items(),
    columns=["Node", "Degree Centrality"]
)

degree_df = degree_df.sort_values(
    by="Degree Centrality",
    ascending=False
)

degree_df
```

	Node	Betweenness Centrality
13	N	0.292245
11	L	0.249769
21	V	0.223137
3	D	0.206531
17	R	0.188999
6	G	0.188189
5	F	0.069965
4	E	0.051849
8	I	0.050696
20	U	0.045130
15	P	0.044735
7	H	0.043187
16	Q	0.039992
12	M	0.034914
10	K	0.033564
19	T	0.023786
18	S	0.018187
2	C	0.011397
1	B	0.008432
9	J	0.008235
14	O	0.006719
22	W	0.006192
0	A	0.000000
23	X	0.000000

```python
closeness = nx.closeness_centrality(G)

closeness_df = pd.DataFrame(
    closeness.items(),
    columns=["Node", "Closeness Centrality"]
)

closeness_df = closeness_df.sort_values(
    by="Closeness Centrality",
    ascending=False
)

closeness_df
```

	Node	Closeness Centrality
13	N	0.469388
11	L	0.469388
3	D	0.410714
10	K	0.410714
21	V	0.410714
6	G	0.403509
12	M	0.396552
17	R	0.396552
5	F	0.383333
19	T	0.377049
8	I	0.370968
7	H	0.370968
4	E	0.370968
18	S	0.370968
15	P	0.370968
16	Q	0.365079
20	U	0.365079
9	J	0.359375
14	O	0.353846
2	C	0.338235
1	B	0.333333
22	W	0.323944
0	A	0.302632
23	X	0.298701

Figure 3. Centrality results: degree, betweenness and closeness.

The results can be used to pinpoint the most critical nodes in the network. Degree centrality indicates those nodes with the most direct connections. The Betweenness centrality is a

measure of nodes that serve as "bridges" between other parts of the network. The central nodes identified by closeness centrality are those that are capable of reaching other nodes efficiently. The node having the highest betweenness centrality can be regarded as one of the most important bridge nodes in the network after the code has been run since it links various parts of the graph.

Community Detection

The community detection was carried out with the greedy modularity method implemented in NetworkX.

The algorithm clustered the network into groups according to density of connections. The nodes in each community are more connected among themselves than to the nodes outside of the community.

The detected communities were then displayed in various colors of nodes.

```python
from networkx.algorithms.community import greedy_modularity_communities

communities = list(
    greedy_modularity_communities(G)
)

for i, community in enumerate(communities, start=1):
    print(f"Community {i}: {sorted(community)}")
```

```
Community 1: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
Community 2: ['I', 'J', 'K', 'L', 'M', 'N', 'O', 'P']
Community 3: ['Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X']
```

```python
community_map = {}

for i, community in enumerate(communities):
    for node in community:
        community_map[node] = i
```

```python
plt.figure(figsize=(12, 8))

pos = nx.spring_layout(G, seed=42)

node_colors = [
    community_map[node]
    for node in G.nodes()
]

nx.draw(
    G,
    pos,
    with_labels=True,
    node_color=node_colors,
    cmap=plt.cm.Set3,
    node_size=900,
    font_size=10
)

plt.title("Community Detection in Social Network")
plt.show()
```

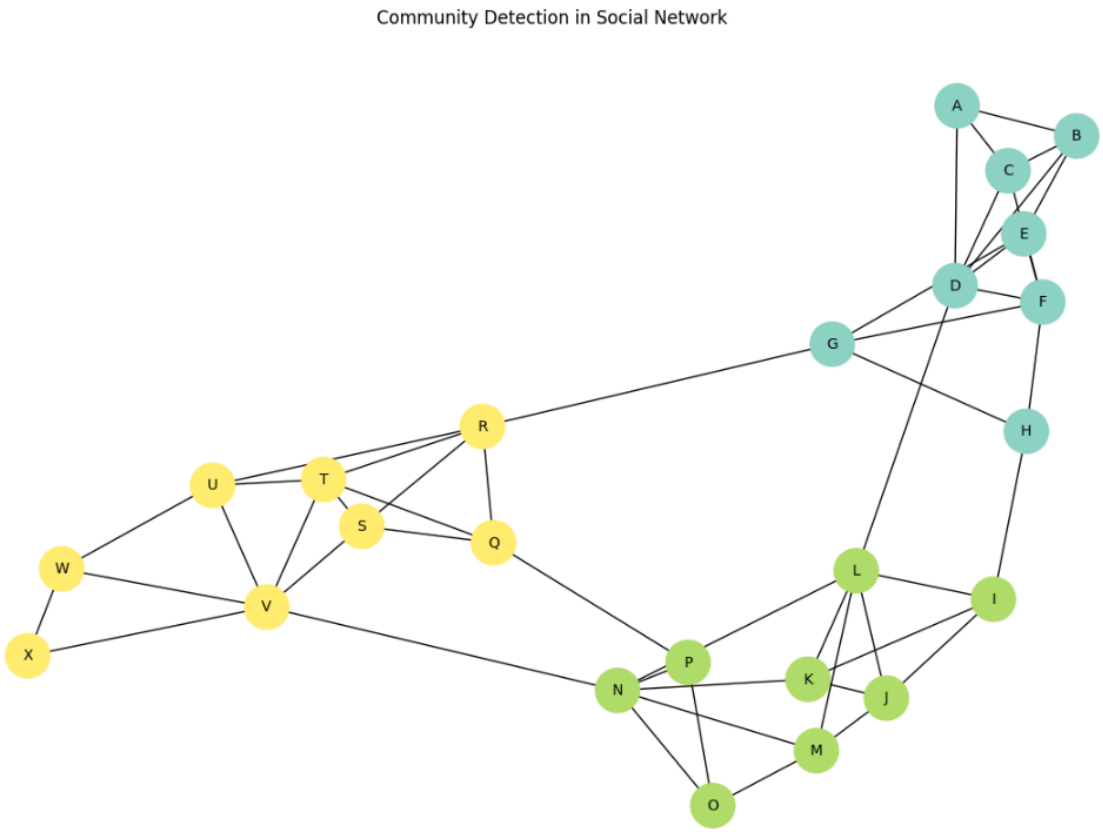


Figure 4. Visualization of communities detected in the social network.

The image depicts several clusters of nodes with strong connections between them, making it clear that there are multiple clusters of nodes (Luo et al., 2008). Some of the connections between the groups also show that a few nodes may serve as bridges between communities.

The centrality and community detection are explored in this relationship.This relationship is explored between centrality and community detection.

Centrality and community detection can be used together to provide a better understanding of a network. Community detection can be used to establish the primary clusters of the network, but centrality can be used to determine which node in each cluster is the most influential. For instance, if a social media company identifies current communities of users, they

might turn to degree or betweenness centrality to discover influential users within each community. This information may be successfully applied to targeted marketing, recommendation systems, communication analysis and network monitoring.

Hypothetical Big Data Application.

The same methods can be used by a large social media firm in a system with millions of members. The users would be the nodes and friendships/follows/interactions would be the edges. Community detection might enable the identification of a cluster of users with similar interaction styles. Centrality measures might detect high degree or high betweenness users or those who serve as 'bridges' between user groups. These results can be utilized in several ways, such as for providing personalized recommendations, advertising, information distribution, and identification of influential users by the company. In cyber security, this approach can also be used to determine the critical devices and clusters of devices.

Conclusion

To sum it up, there are many useful tools for graph analytics and network science called centrality measures, and community detection is one such tool. The measures of centrality and community detection can be used to identify important and influential nodes, and to reveal hidden groups within the network. To implement, Python and NetworkX were used in Google Colab to build and explore a hypothetical social network. Graph Traversal showed traversals of the graph, Centrality measures identified important nodes, and Community detection showed groups of closely connected nodes. The practical results are directly related to the theoretical consideration and illustrate the application of graph analytics in real world problems of big data.

References

Cheramangalath, U., Nasre, R., & Srikant, Y. N. (2020). Graph Analytics Frameworks. In *Springer eBooks* (pp. 99–122). https://doi.org/10.1007/978-3-030-41886-1_4

Hagberg, A. [. A. N. L. L. a. N. S. P. J. [. A. N. L. L. a. N. S. D. a. [. U. H. N. (2024). Exploring network structure, dynamics, and function using NetworkX. *OSTI OAI (U.S. Department of Energy Office of Scientific and Technical Information)*. https://www.osti.gov/biblio/960616

Jiang, B. (1991). Traversing graphs in a paging environment, BFS or DFS? *Information Processing Letters*, *37*(3), 143–147. https://doi.org/10.1016/0020-0190(91)90034-f

Luo, F., Wang, J. Z., & Promislow, E. (2008). Exploring local community structures in large networks. *Web Intelligence and Agent Systems an International Journal*, *6*(4), 387–400. https://doi.org/10.3233/wia-2008-0147